

PRDV2.0

Luminous Stays- Maui Hospitality

Property & Audience:

Luminous Stays is a luxury oceanfront vacation rental brand that is located on the beautiful Island of Maui, Hawaii with additional properties in Wailea, South Kihei, Hana, Lahaina, and Paia. The platform is designed for travelers that are seeking high end experience, specifically honeymooners, couples, and retirees who value privacy, comfort, and an oceanfront experience. The application includes a Marketing Page, Property Dashboard, and Admin Login. The goal is to connect a marketing website with backend and an admin management system.

Problem Statement:

Luminous Stays exist to make it easier for people to find a luxury oceanfront rental in Maui and to help the property owner understand tourism trends. Right now, travel information is scattered and it is hard for visitors to know what they are actually getting. At the same time, owners don't always have clear data on demand and visitor patterns. This app connects both sides by combining a marketing site with a dashboard that shows real data and insights.

User Personas:

Visitor A: Jake and Aida Stevenson are newlyweds from Canada that are looking for a luxury oceanfront stay with amenities and nearby attractions on Maui.

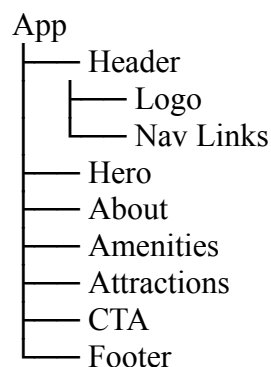
Visitor B: Robert & Lisa Smith are both retired and celebrating their 50th anniversary. They are looking for a comfortable, private, and relaxing oceanfront two-week stay on Maui.

Admin: Aisha Patterson is the property owner managing listings, monitoring data, and uses a secure admin dashboard to update and maintain the platform.

Marketing Page:

- Hero: Displays the property name, tagline, & tropical Maui background image.
- About: Shows the property description and location using data from GET/properties/id.
- Amenities: Displays key features such WiFi, Pool, Gym, Ocean View, & Parking from MongoDB.
- Attractions: Highlights nearby locations like Road to Hana, Haleakala, Wailea Beach, & local food spots.
- CTA: Includes a booking button to encourage guest inquiries.

Data Architecture:



- MongoDB stores the property data and reviews.
- Express API provides GET /properties, GET /properties/:id , and POST /properties/:id/reviews.
- React uses useEffect to fetch data and useState to store it.
- External APIs include OpenWeatherMap and DBEDT tourism data
- DBEDT data includes visitor arrivals, length of stay, and accommodation type by island and month. The data will be converted to JSON & served through the GET/api/tourism route on the Express server.

Acceptance Criteria Metrics:

- Given the page loads, the Hero section displays the Luminous Stays name and background within 3 seconds.
- Given the amenities load, all the data appears correctly from the database within 2 seconds.
- Given the admin login succeeds, the user is redirected to the dashboard within 1 second.
- Given the property page loads, all of the details are displayed correctly including the reviews within 2 seconds.
- When a review is submitted, it is saved to MongoDB and shown in the UI within 1 second.
- Given someone is not logged in, when they try to go to /admin/dashboard, they get sent to /admin/login and can't see any admin stuff.
- Given a user with a Google account, when they click "Sign in with Google" and approve it, they get redirected to the dashboard and their info gets saved in MongoDB with googleId and provider set as google.
- Given the GitHub repo, when anyone checks it, the .env file isn't there, but .env.example is, and there's no API keys showing anywhere.

Dashboard Design:

I will build three data visualizations using [Chart.js](#):

- Bar chart: Shows Maui visitor arrivals by month using DBEDT data. This helps identify peak travel seasons so both guests & property owners can plan bookings & pricing more effectively.
- Pie chart: Shows the breakdown of visitor types (Hotel vs Vacation rental). This helps the owner understand what types of accommodation travelers are choosing in Maui.
- Line Chart: Shows average length of stay trends over time. This helps show booking behavior, especially for couples & retirees planning longer stays.

Island Selector UX:

I will use a dropdown menu to let users filter dashboard data by island. The selected island will be stored in the useState variable called selectedIsland. When the selection changes, the charts and dashboard data will update to show only that island results. The default selection will be Maui because that is where Luminous Stays is located.

Weather Widget:

I will add a live weather widget using OpenWeatherMapAPI. It will be placed at the top of the dashboard so guests can quickly see current Maui weather while planning their trip. The widget will display the temperature, weather conditions, and humidity. The city used for the API request will be Wailea, Maui. If the data is still loading, a simple “Loading weather...” message will appear until the weather information is ready.

Week 14-15 plans:

In week 14, I will build the admin login page with the username and password authentication using [Passport.js](#). Only authenticated admin users will be able to access the protected dashboard to update listings and post special announcements. Passwords will be stored securely using bcrypt.

In week 15, I set up Google OAuth 2.0 as another way to log in using passport-google-oauth20. I also added [Helmet.js](#) for extra security headers, ran final tests to make sure everything works and got the app ready to deploy on Render so it can finally be live and accessible to everyone.

Authentication Requirements:

The admin login uses email and password with [Passport.js](#) LocalStrategy. The admin is the property owner and they are the only one who can access the dashboard, update listings, and manage the site.

Protected routes:

- /admin/login - login page
- /admin/dashboard - protected dashboard

Login flow:

1. The admin goes to /admin/login and enters their email and password
2. Passport checks the email in MongoDB
3. Bcrypt compares the password with the stored hash
4. If it matches, the user is logged in and a session is created
5. The admin is redirected to the dashboard
6. The session keeps them logged in until they log out

When the admin logs out or closes the browser, the session ends and they have to login again.

Security Requirements:

Passwords are hashed using bcrypt with 10 salt rounds, so they are not stored as plain text. This is important because if the database gets compromised, the real passwords are not exposed. Important values like SESSION_SECRET, MONGODB_URI, and VITE_WEATHER_KEY are stored in a .env file. This file is not pushed to GitHub because it is listed in .gitignore. Instead, a .env.example file is included. If a login fails, the user is sent back to the login page with an error message instead of the app crashing. One possible risk is users entering ad input in forms, like reviews. This could lead to XSS attacks. To prevent this, I will add input validation using express-validator.

User Stories:

1. As an admin, I want to login with my email and password so I can manage listings and special offers.
2. As a visitor, I should not be able to access the admin dashboard so the system stays secure.

AI Attribution:

AI tools (Claude) were used to support understanding and helping me with organizing and refining this document. The project concept and design decisions are my own.

PRDV2.0**Luminous Stays- Maui Hospitality****Property & Audience:**

Luminous Stays is a luxury oceanfront vacation rental brand that is located on the beautiful Island of Maui, Hawaii with additional properties in Wailea, South Kihei, Hana, Lahaina, and Paia. The platform is designed for travelers that are seeking high end experience, specifically honeymooners, couples, and retirees who value privacy, comfort, and an oceanfront experience. The application includes a Marketing Page, Property Dashboard, and Admin Login. The goal is to connect a marketing website with backend and an admin management system.

Problem Statement:

Luminous Stays exist to make it easier for people to find a luxury oceanfront rental in Maui and to help the property owner understand tourism trends. Right now, travel information is scattered and it is hard for visitors to know what they are actually getting. At the same time, owners don't always have clear data on demand and visitor patterns. This app connects both sides by combining a marketing site with a dashboard that shows real data and insights.

User Personas:

Visitor A: Jake and Aida Stevenson are newlyweds from Canada that are looking for a luxury oceanfront stay with amenities and nearby attractions on Maui.

Visitor B: Robert & Lisa Smith are both retired and celebrating their 50th anniversary. They are looking for a comfortable, private, and relaxing oceanfront two-week stay on Maui.

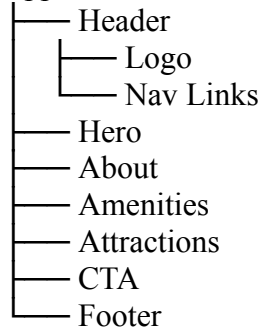
Admin: Aisha Patterson is the property owner managing listings, monitoring data, and uses a secure admin dashboard to update and maintain the platform.

Marketing Page:

- Hero: Displays the property name, tagline, & tropical Maui background image.
- About: Shows the property description and location using data from GET/properties/id.
- Amenities: Displays key features such WiFi, Pool, Gym, Ocean View, & Parking from MongoDB.
- Attractions: Highlights nearby locations like Road to Hana, Haleakala, Wailea Beach, & local food spots.
- CTA: Includes a booking button to encourage guest inquiries.

Data Architecture:

App



- MongoDB stores the property data and reviews.
- Express API provides GET /properties, GET /properties/:id , and POST /properties/:id/reviews.
- React uses useEffect to fetch data and useState to store it.
- External APIs include OpenWeatherMap and DBEDT tourism data
- DBEDT data includes visitor arrivals, length of stay, and accommodation type by island and month. The data will be converted to JSON & served through the GET/api/tourism route on the Express server.

Acceptance Criteria Metrics:

- Given the page loads, the Hero section displays the Luminous Stays name and background within 3 seconds.
- Given the amenities load, all the data appears correctly from the database within 2 seconds.
- Given the admin login succeeds, the user is redirected to the dashboard within 1 second.
- Given the property page loads, all of the details are displayed correctly including the reviews within 2 seconds.
- When a review is submitted, it is saved to MongoDB and shown in the UI within 1 second.
- Given someone is not logged in, when they try to go to /admin/dashboard, they get sent to /admin/login and can't see any admin stuff.
- Given a user with a Google account, when they click "Sign in with Google" and approve it, they get redirected to the dashboard and their info gets saved in MongoDB with googleId and provider set as google.
- Given the GitHub repo, when anyone checks it, the .env file isn't there, but .env.example is, and there's no API keys showing anywhere.

Dashboard Design:

I will build three data visualizations using [Chart.js](#):

- Bar chart: Shows Maui visitor arrivals by month using DBEDT data. This helps identify peak travel seasons so both guests & property owners can plan bookings & pricing more effectively.
- Pie chart: Shows the breakdown of visitor types (Hotel vs Vacation rental). This helps the owner understand what types of accommodation travelers are choosing in Maui.
- Line Chart: Shows average length of stay trends over time. This helps show booking behavior, especially for couples & retirees planning longer stays.

Island Selector UX:

I will use a dropdown menu to let users filter dashboard data by island. The selected island will be stored in the useState variable called selectedIsland. When the selection changes, the charts and dashboard data will update to show only that island results. The default selection will be Maui because that is where Luminous Stays is located.

Weather Widget:

I will add a live weather widget using OpenWeatherMapAPI. It will be placed at the top of the dashboard so guests can quickly see current Maui weather while planning their trip. The widget will display the temperature, weather conditions, and humidity. The city used for the API request will be Wailea, Maui. If the data is still loading, a simple “Loading weather...” message will appear until the weather information is ready.

Week 14-15 plans:

In week 14, I will build the admin login page with the username and password authentication using [Passport.js](#). Only authenticated admin users will be able to access the protected dashboard to update listings and post special announcements. Passwords will be stored securely using bcrypt.

In week 15, I set up Google OAuth 2.0 as another way to log in using passport-google-oauth20. I also added [Helmet.js](#) for extra security headers, ran final tests to make sure everything works and got the app ready to deploy on Render so it can finally be live and accessible to everyone.

Authentication Requirements:

The admin login uses email and password with [Passport.js](#) LocalStrategy. The admin is the property owner and they are the only one who can access the dashboard, update listings, and manage the site.

Protected routes:

- /admin/login - login page
- /admin/dashboard - protected dashboard

Login flow:

7. The admin goes to /admin/login and enters their email and password
8. Passport checks the email in MongoDB
9. Bcrypt compares the password with the stored hash
10. If it matches, the user is logged in and a session is created
11. The admin is redirected to the dashboard
12. The session keeps them logged in until they log out

When the admin logs out or closes the browser, the session ends and they have to login again.

Security Requirements:

Passwords are hashed using bcrypt with 10 salt rounds, so they are not stored as plain text. This is important because if the database gets compromised, the real passwords are not exposed. Important values like SESSION_SECRET, MONGODB_URI, and VITE_WEATHER_KEY are stored in a .env file. This file is not pushed to GitHub because it is listed in .gitignore. Instead, a .env.example file is included. If a login fails, the user is sent back to the login page with an error message instead of the app crashing. One possible risk is users entering ad input in forms, like reviews. This could lead to XSS attacks. To prevent this, I will add input validation using express-validator.

User Stories:

3. As an admin, I want to login with my email and password so I can manage listings and special offers.
4. As a visitor, I should not be able to access the admin dashboard so the system stays secure.

Test Scripts:

```
// tests/auth.test.js
const request = require('supertest');
const app = require('../app');
// AC-6: unauthenticated user redirected from dashboard
test('GET /admin/dashboard redirects to /admin/login when not logged in', async () => {
  const res = await request(app).get('/admin/dashboard');
  expect(res.statusCode).toBe(302);
  expect(res.headers.location).toBe('/admin/login');
});
// AC-7: logout clears session
test('POST /logout redirects to login page', async () => {
  const res = await request(app).post('/logout');
  expect(res.statusCode).toBe(302);
  expect(res.headers.location).toBe('/admin/login');
});
// AC-1: marketing page loads successfully
test('GET / returns 200 and renders marketing page', async () => {
```

```
const res = await request(app).get('/');  
expect(res.statusCode).toBe(200);  
});
```

AI Attribution:

AI tools (Claude) were used to support understanding and helping me with organizing and refining this document. The project concept and design decisions are my own.